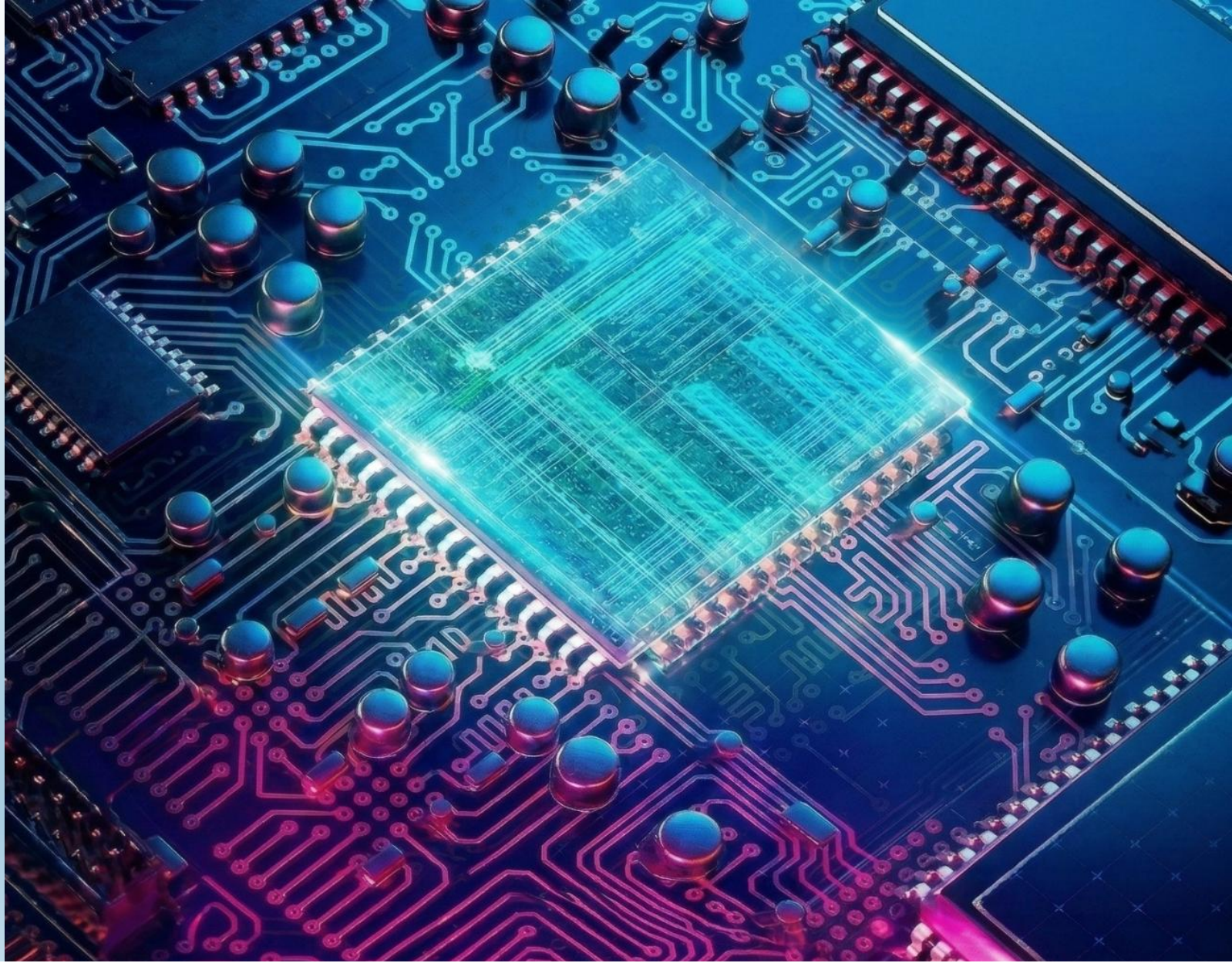




Digital Verification Assignment 5

Name: Mohamed Torki

Alexandria University, Faculty of Engineering
Electronics and Communications Department

Part 1:

Interface (The Same all over 3 Parts):

```
interface alsu_if (clk);
    input clk;
    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
    logic [2:0] opcode;
    logic signed [2:0] A, B;
    logic [15:0] leds;
    logic signed [5:0] out;
endinterface
```

Environment:

```
package alsu_env_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

class alsu_env extends uvm_env;
    `uvm_component_utils(alsu_env)
    function new(string name = "alsu_env", uvm_component parent = null);
        super.new(name, parent);
    endfunction
endclass
endpackage
```



Test:

```
package alsu_test_pkg;
import uvm_pkg::*;
import alsu_env_pkg::*;
`include "uvm_macros.svh"
class alsu_test extends uvm_test;
    `uvm_component_utils(alsu_test)

    alsu_env env;

    function new (string name = "alsu_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = alsu_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);
        #100; `uvm_info("alsu_test", "Inside the ALSU Test.", UVM_NONE);
        phase.drop_objection(this);
    endtask
endclass

endpackage
```



Top Module:

```
import uvm_pkg::*;
`include "uvm_macros.svh"
import alsu_test_pkg::*;

module top();

    bit clk;
    initial begin
        clk = 0;
        forever
            #1 clk = ~clk;
    end

    alsu_if alsuif(clk);

    ALSU DUT (
        .clk      (alsuif.clk),
        .rst      (alsuif.rst),
        .cin      (alsuif.cin),
        .serial_in (alsuif.serial_in),
        .red_op_A  (alsuif.red_op_A),
        .red_op_B  (alsuif.red_op_B),
        .opcode    (alsuif.opcode),
        .bypass_A  (alsuif.bypass_A),
        .bypass_B  (alsuif.bypass_B),
        .direction (alsuif.direction),
        .A         (alsuif.A),
        .B         (alsuif.B),
        .leds      (alsuif.leds),
        .out       (alsuif.out)
    );

    initial begin
        run_test("alsu_test");
    end
endmodule
```



Sources File:

```

Assignments > A5 > Part_1 > src_files.list
1  ALSU.v
2  alsu_if.sv
3  alsu_env.sv
4  alsu_test.sv
5  top.sv

```

Do File:

```

Assignments > A5 > Part_1 > run.do
1  vlib work
2  vlog -f src_files.list
3  vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
4  run -all

```

Message:

```

Transcript
# (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test alsu_test...
# UVM_INFO alsu_test.sv(22) @ 100: uvm_test_top [alsu_test] Inside the ALSU Test.
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 5
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
#
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [alsu_test] 1
#
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 100 ns Iteration: 54 Instance: /top

```



Part 2:

Driver:

```

package alsu_driver_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

class alsu_driver extends uvm_driver;

    virtual alsu_if alsu_driver_vif;

    `uvm_component_utils(alsu_driver)

    function new(string name = "alsu_driver", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual alsu_if)::get(this, "", "ALSU_IF",
alsu_driver_vif))
            `uvm_fatal("build_phase","Unable to get Virtual Interface to the
driver");
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        alsu_driver_vif.rst = 1;
        alsu_driver_vif.A = 0;
        alsu_driver_vif.B = 0;
        alsu_driver_vif.cin = 0;
        alsu_driver_vif.red_op_A = 0;
        alsu_driver_vif.red_op_B = 0;
        alsu_driver_vif.bypass_A = 0;
        alsu_driver_vif.bypass_B = 0;
        alsu_driver_vif.direction = 0;
        alsu_driver_vif.serial_in = 0;
        alsu_driver_vif.opcode = 0;

        @(negedge alsu_driver_vif.clk);
        alsu_driver_vif.rst = 0;
        forever begin
            @(negedge alsu_driver_vif.clk);

```



```

        alsu_driver_vif.A = $random;
        alsu_driver_vif.B = $random;
        alsu_driver_vif.cin = $random;
        alsu_driver_vif.red_op_A = $random;
        alsu_driver_vif.red_op_B = $random;
        alsu_driver_vif.bypass_A = $random;
        alsu_driver_vif.bypass_B = $random;
        alsu_driver_vif.direction = $random;
        alsu_driver_vif.serial_in = $random;
        alsu_driver_vif.opcode = $random;
        alsu_driver_vif.rst = $random;
    end
endtask
endclass

endpackage

```

Environment:

```

package alsu_env_pkg;
import uvm_pkg::*;
import alsu_driver_pkg::*;
`include "uvm_macros.svh"

alsu_driver driver;

class alsu_env extends uvm_env;
    `uvm_component_utils(alsu_env)
    function new(string name = "alsu_env", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        driver = alsu_driver::type_id::create("driver", this);
    endfunction:build_phase
endclass

endpackage

```



Test:

```

package alsu_test_pkg;
import uvm_pkg::*;
import alsu_env_pkg::*;
`include "uvm_macros.svh"
class alsu_test extends uvm_test;
    `uvm_component_utils(alsu_test)

    alsu_env env;
    virtual alsu_if alsu_test_vif;

    function new (string name = "alsu_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual alsu_if)::get(this, "", "ALSU_IF",
alsu_test_vif))
            `uvm_fatal("build_phase", "Unable to get Virtual Interface");
        uvm_config_db#(virtual alsu_if)::set(this, "*", "ALSU_IF",
alsu_test_vif);
        env = alsu_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);
        #100; `uvm_info("alsu_test", "Inside the ALSU Test.", UVM_NONE);
        phase.drop_objection(this);
    endtask
endclass

endpackage

```



Top Module:

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import alsu_test_pkg::*;

module top();

    bit clk;
    initial begin
        clk = 0;
        forever
            #1 clk = ~clk;
    end

    alsu_if alsuif(clk);

    ALSU DUT ( .clk      (alsuif.clk),
               .rst      (alsuif.rst),
               .cin      (alsuif.cin),
               .serial_in (alsuif.serial_in),
               .red_op_A  (alsuif.red_op_A),
               .red_op_B  (alsuif.red_op_B),
               .opcode    (alsuif.opcode),
               .bypass_A  (alsuif.bypass_A),
               .bypass_B  (alsuif.bypass_B),
               .direction (alsuif.direction),
               .A          (alsuif.A),
               .B          (alsuif.B),
               .leds       (alsuif.leds),
               .out        (alsuif.out)
               );

    initial begin
        uvm_config_db#(virtual alsu_if)::set(null, "uvm_test_top", "ALSU_IF",
alsuif);
        run_test("alsu_test");
    end
endmodule

```



Part 3:

Configuration Object:

```
package alsu_config_obj_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
class alsu_config_obj extends uvm_object;
    `uvm_object_utils(alsu_config_obj)

    virtual alsu_if alsu_config_vif;

    function new(string name = "alsu_config_obj");
        super.new(name);
    endfunction
endclass
endpackage
```



Driver:

```

package alsu_driver_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
import alsu_config_obj_pkg::*;
    class alsu_driver extends uvm_driver;

        virtual alsu_if alsu_driver_vif;
        alsu_config_obj alsu_config_obj_driver;

        `uvm_component_utils(alsu_driver)

        function new(string name = "alsu_driver", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            if(!uvm_config_db#(alsu_config_obj)::get(this, "", "CFG",
alsu_config_obj_driver))
                `uvm_fatal("build_phase","Unable to get Configuration Object to
the driver");
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            alsu_driver_vif = alsu_config_obj_driver.alsu_config_vif;
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            alsu_driver_vif.rst = 1;
            alsu_driver_vif.A = 0;
            alsu_driver_vif.B = 0;
            alsu_driver_vif.cin = 0;
            alsu_driver_vif.red_op_A = 0;
            alsu_driver_vif.red_op_B = 0;
            alsu_driver_vif.bypass_A = 0;
            alsu_driver_vif.bypass_B = 0;
            alsu_driver_vif.direction = 0;
            alsu_driver_vif.serial_in = 0;
            alsu_driver_vif.opcode = 0;

            @(negedge alsu_driver_vif.clk);

```



```

        alsu_driver_vif.rst = 0;
        forever begin
            @(negedge alsu_driver_vif.clk);
            alsu_driver_vif.A = $random;
            alsu_driver_vif.B = $random;
            alsu_driver_vif.cin = $random;
            alsu_driver_vif.red_op_A = $random;
            alsu_driver_vif.red_op_B = $random;
            alsu_driver_vif.bypass_A = $random;
            alsu_driver_vif.bypass_B = $random;
            alsu_driver_vif.direction = $random;
            alsu_driver_vif.serial_in = $random;
            alsu_driver_vif.opcode = $random;
            alsu_driver_vif.rst = $random;
        end
    endtask
endclass

endpackage

```

Environment:

```

package alsu_env_pkg;
import uvm_pkg::*;
import alsu_driver_pkg::*;
`include "uvm_macros.svh"

class alsu_env extends uvm_env;
    alsu_driver driver;
    `uvm_component_utils(alsu_env)
    function new(string name = "alsu_env", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        driver = alsu_driver::type_id::create("driver", this);
    endfunction:build_phase
endclass

endpackage

```



Test:

```

package alsu_test_pkg;
import uvm_pkg::*;
import alsu_env_pkg::*;
import alsu_config_obj_pkg::*;
`include "uvm_macros.svh"

class alsu_test extends uvm_test;
    `uvm_component_utils(alsu_test)

    alsu_env env;
    alsu_config_obj alsu_config_obj_test;

    function new (string name = "alsu_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = alsu_env::type_id::create("env", this);
        alsu_config_obj_test =
alsu_config_obj::type_id::create("alsu_config_obj_test");
        if(!uvm_config_db#(virtual alsu_if)::get(this, "", "ALSU_IF",
alsu_config_obj_test.alsu_config_vif))
            `uvm_fatal("build_phase", "Unable to get Virtual Interface from
the Configuration Object");
        uvm_config_db#(alsu_config_obj)::set(this, "*", "CFG",
alsu_config_obj_test);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);
        #100; `uvm_info("alsu_test", "Inside the ALSU Test.", UVM_NONE);
        phase.drop_objection(this);
    endtask
endclass

endpackage

```



Top Module:

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import alsu_test_pkg::*;

module top();

    bit clk;
    initial begin
        clk = 0;
        forever
            #1 clk = ~clk;
    end

    alsu_if alsuif(clk);

    ALSU DUT ( .clk      (alsuif.clk),
               .rst      (alsuif.rst),
               .cin      (alsuif.cin),
               .serial_in (alsuif.serial_in),
               .red_op_A  (alsuif.red_op_A),
               .red_op_B  (alsuif.red_op_B),
               .opcode    (alsuif.opcode),
               .bypass_A  (alsuif.bypass_A),
               .bypass_B  (alsuif.bypass_B),
               .direction (alsuif.direction),
               .A          (alsuif.A),
               .B          (alsuif.B),
               .leds       (alsuif.leds),
               .out        (alsuif.out)
               );

    initial begin
        uvm_config_db#(virtual alsu_if)::set(null, "uvm_test_top", "ALSU_IF",
alsuif);
        run_test("alsu_test");
    end
endmodule

```



Sources File:

```

Assignments > A5 > Part_3 > src_files.list
1  ALSU.v
2  alsu_if.sv
3  alsu_config_obj.sv
4  alsu_driver.sv
5  alsu_env.sv
6  alsu_test.sv
7  top.sv

```

Do File:

```

Assignments > A5 > Part_3 > run.do
1  vlib work
2  vlog -f src_files.list
3  vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
4  add wave -position insertpoint sim:/top/alsuif/*
5  run -all

```

Message:

```

Transcript
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test alsu_test...
# UVM_INFO alsu_test.sv(28) @ 100: uvm_test_top [alsu_test] Inside the ALSU Test.
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
--- UVM Report Summary ---
#
** Report counts by severity
# UVM_INFO : 5
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
#
** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [alsu_test] 1
#
** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 100 ns Iteration: 54 Instance: /top

```

Waveform:

